🚀 Projeto: Plataforma de Monitoramento de Data Drift para Detecção de Fraude

📋 Visão Geral do Projeto

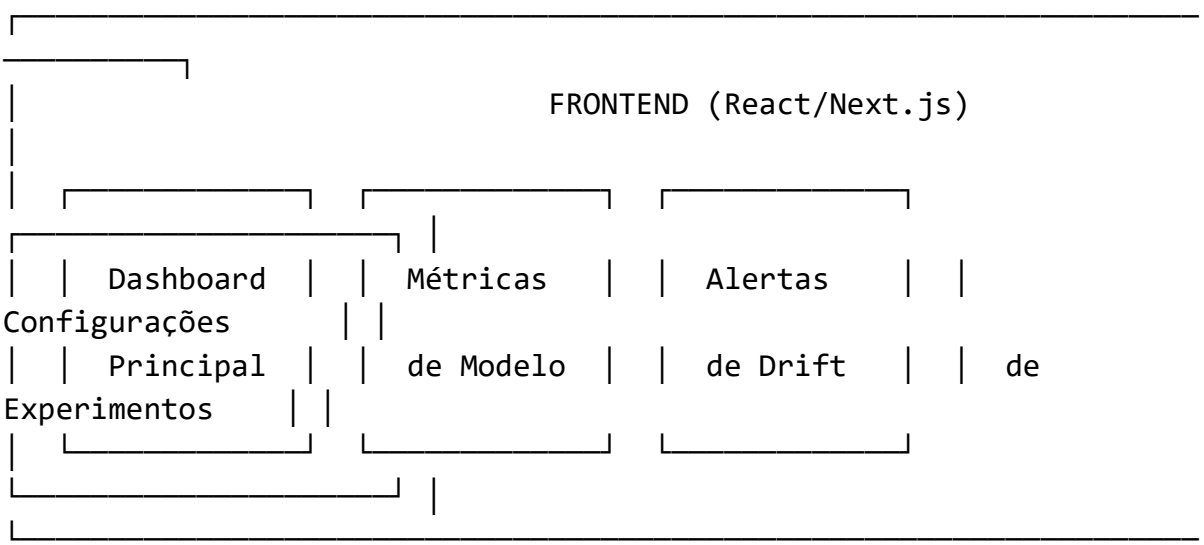
O projeto consiste em uma **plataforma experimental** que simula o ambiente de um banco ou fintech, integrando:

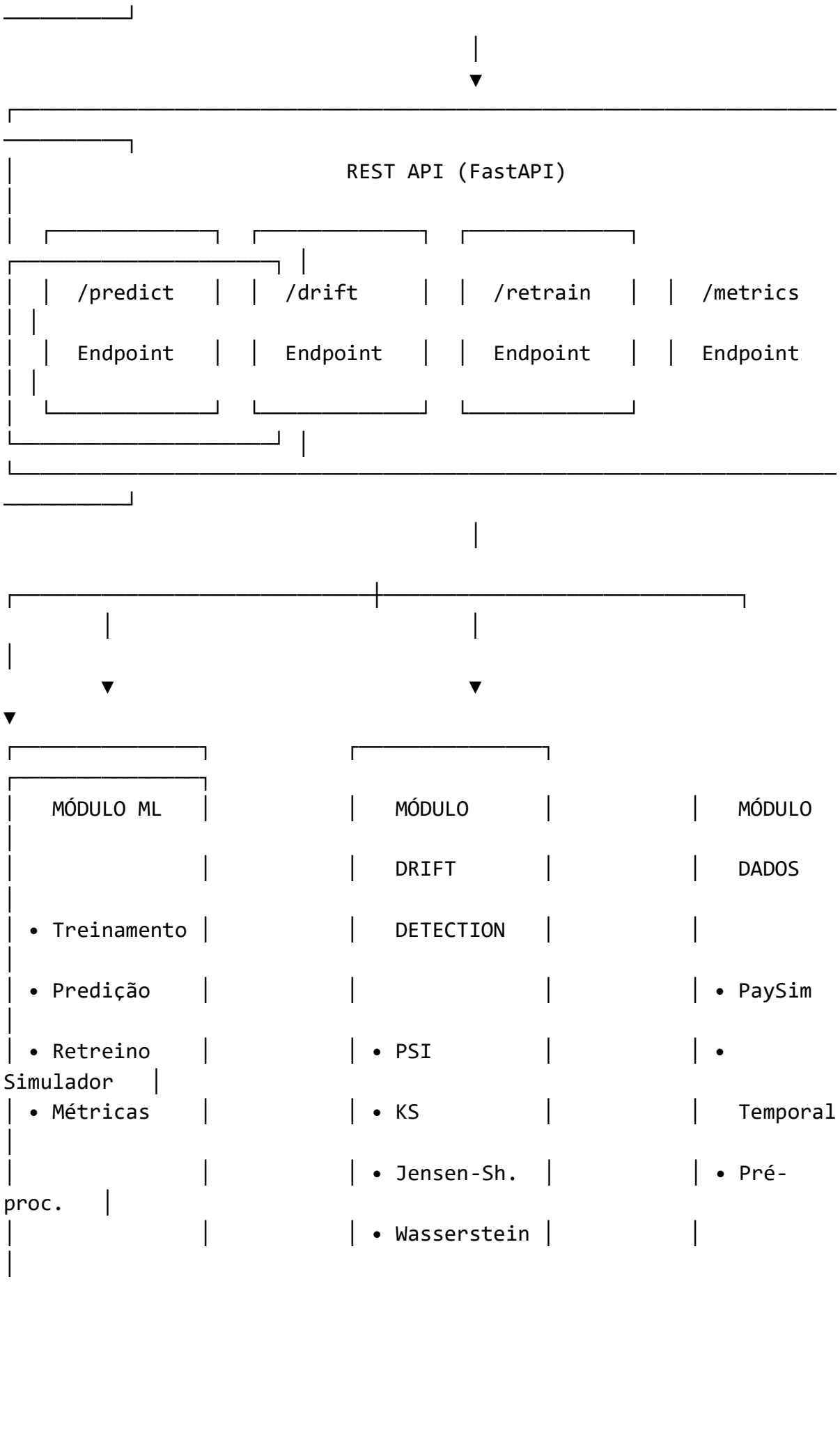
1. **Modelo de Machine Learning** para detecção de fraudes
2. **Simulador de passagem do tempo** com diferentes níveis de Data Drift
3. **Sistema de detecção de drift** com múltiplos métodos estatísticos
4. **Dashboard de monitoramento** para visualização e alertas
5. **Mecanismo de retreinamento** orientado por alertas

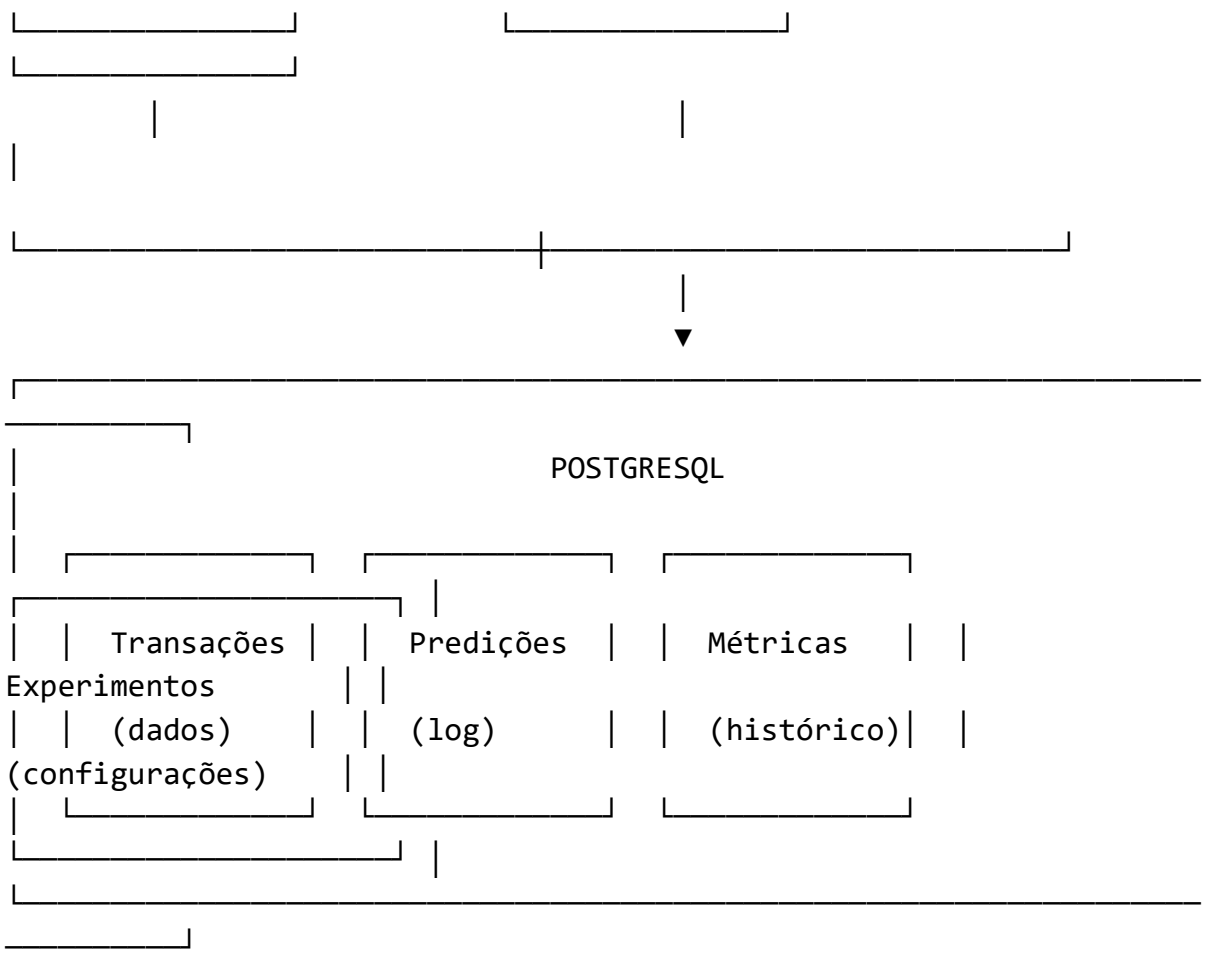
Objetivo científico: Servir como instrumento experimental para investigar o impacto do Data Drift no desempenho de modelos de detecção de fraude e a eficácia de estratégias de mitigação.

🏗️ ARQUITETURA DO SISTEMA

Diagrama Geral da Arquitetura




REST API (FastAPI)
/predict
Endpoint
/drift
Endpoint
/retrain
Endpoint
/metrics
Endpoint
MÓDULO ML
• Treinamento
• Predição
• Retreino
• Métricas
MÓDULO DRIFT DETECTION
• PSI
• KS
• Jensen-Sh.
• Wasserstein
MÓDULO DADOS
• PaySim
• Simulador Temporal
• Pré-proc.



👥 DIVISÃO DE TAREFAS PARA 3 PESSOAS

📌 Pessoa 1: Engenheiro(a) de Machine Learning & Dados

Responsabilidades principais:

- Preparação e análise do dataset PaySim
- Desenvolvimento e treinamento do modelo de detecção de fraude
- Implementação do simulador de passagem do tempo e Data Drift
- Avaliação de métricas de desempenho do modelo

Tarefa	Descrição	Entregável	Prazo
T1.1	Análise exploratória do dataset PaySim	Notebook com EDA	Semana 1

T1.2	Pré-processamento e engenharia de features	Pipeline de pré-processamento	Semana 1
T1.3	Treinamento do modelo baseline (XGBoost)	Modelo treinado + métricas iniciais	Semana 2
T1.4	Implementação do simulador temporal (janelas)	Módulo `temporal_simulator.py`	Semana 2
T1.5	Implementação dos cenários de drift (baixo, moderado, forte)	Módulo `drift_simulator.py`	Semana 3
T1.6	Avaliação do modelo nos diferentes cenários	Relatório de métricas (E1, E2)	Semana 3
T1.7	Implementação do retreinamento orientado por alertas	Módulo `retrain.py`	Semana 4
T1.8	Integração com API (endpoints de predição e retreino)	Endpoints funcionais	Semana 4

Stack:

- Python, Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib, Seaborn, Jupyter

📌 Pessoa 2: Engenheiro(a) de Detecção de Drift & Estatística

Responsabilidades principais:

- Implementação dos métodos estatísticos de detecção de drift
- Definição de limiares e sistema de alertas
- Comparação da sensibilidade dos métodos
- Análise estatística dos resultados

Tarefa	Descrição	Entregável	Prazo
T2.1	Estudo e implementação do PSI	Módulo `psi.py`	Semana 1
T2.2	Estudo e implementação do teste KS	Módulo `ks_test.py`	Semana 1

T2.3	Estudo e implementação da divergência Jensen-Shannon	Módulo `js_divergence.py`	Semana 2
T2.4	Estudo e implementação da distância de Wasserstein	Módulo `wasserstein.py`	Semana 2
T2.5	Definição de limiares e sistema de alertas	Módulo `alert_system.py`	Semana 3
T2.6	Comparação da sensibilidade dos métodos (E3)	Relatório comparativo	Semana 3
T2.7	Integração com API (endpoint de drift)	Endpoint funcional	Semana 4
T2.8	Análise estatística dos resultados experimentais	Relatório de análise	Semana 4

Stack:

- Python, SciPy, NumPy, Pandas, Evidently, Matplotlib, Jupyter

📌 Pessoa 3: Engenheiro(a) de Software & Fullstack

Responsabilidades principais:

- Desenvolvimento da API REST (FastAPI)
- Modelagem e implementação do banco de dados (PostgreSQL)
- Desenvolvimento do dashboard (React/Next.js)
- Containerização e documentação do projeto

Tarefa	Descrição	Entregável	Prazo
T3.1	Modelagem do banco de dados (esquema)	Diagrama ER + migrations	Semana 1
T3.2	Configuração do ambiente Docker	`docker-compose.yml`	Semana 1
T3.3	Desenvolvimento da API REST (FastAPI)	Endpoints base	Semana 2

T3.4	Integração da API com módulos ML e Drift	API funcional integrada	Semana 3
T3.5	Desenvolvimento do dashboard (React/Next.js)	Dashboard v1	Semana 3
T3.6	Implementação de gráficos e visualizações	Gráficos de drift e métricas	Semana 4
T3.7	Sistema de alertas visuais	Notificações no dashboard	Semana 4
T3.8	Documentação técnica e README	Documentação completa	Semana 4

Stack:

- Python, FastAPI, PostgreSQL, SQLAlchemy, React, Next.js, Chart.js/Recharts, Docker, Git

🗓️ CRONOGRAMA CONSOLIDADO

Semana	Pessoa 1 (ML/Dados)	Pessoa 2 (Drift/Estatística)	Pessoa 3 (Software/Fullstack)
1	EDA + Pré-processamento	Implementação PSI + KS	Modelagem BD + Docker
2	Treinamento + Simulador temporal	Implementação JS + Wasserstein	API base + Endpoints
3	Cenários de drift + Avaliação	Limiares + Comparação	Integração + Dashboard v1
4	Retreinamento + Integração	Análise estatística + Integração	Gráficos + Documentação

🔬 EXPERIMENTOS CIENTÍFICOS

Estrutura dos Experimentos

Experimento	Objetivo	Responsável Principal	Métricas
E1	Medir desempenho sem Data Drift	Pessoa 1	Accuracy, Precision, Recall, F1, AUC-ROC
E2	Introduzir diferentes níveis de Data Drift	Pessoa 1	Mesmas métricas + comparação temporal
E3	Comparar métodos de detecção	Pessoa 2	Sensibilidade, Especificidade, Latência
E4	Avaliar se retreinamento recupera desempenho	Pessoa 1 + Pessoa 2	Recuperação de F1, tempo de recuperação

Variáveis do Estudo

Tipo	Variável	Descrição
Independente	Grau de Data Drift	Baixo, Moderado, Forte
Dependentes	Métricas de desempenho	Accuracy, Precision, Recall, F1, AUC-ROC
Dependentes	Métricas de detecção	PSI, KS, JS, Wasserstein
Dependentes	Tempo de detecção	Janelas até alerta
Controle	Modelo	XGBoost (fixo)
Controle	Dataset	PaySim (fixo)
Controle	Hiperparâmetros	Definidos no baseline

🗀 ESTRUTURA DE DIRETÓRIOS DO PROJETO

```
data-drift-fraud-monitor/
│
├── backend/
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py                 # FastAPI app
│   │   ├── routes/
│   │   │   ├── predict.py          # Endpoint de predição
│   │   │   ├── drift.py            # Endpoint de detecção de drift
│   │   │   ├── retrain.py          # Endpoint de retreinamento
│   │   │   └── metrics.py          # Endpoint de métricas
│   │   └── schemas/
│   │       ├── transaction.py      # Schema de transação
│   │       └── drift.py            # Schema de drift
│   │
│   ├── ml/
│   │   ├── __init__.py
│   │   ├── model.py                # Definição do modelo XGBoost
│   │   ├── train.py                # Treinamento
│   │   ├── predict.py              # Predição
│   │   ├── retrain.py              # Retreinamento
│   │   └── metrics.py              # Métricas de desempenho
│   │
│   ├── drift/
│   │   ├── __init__.py
│   │   ├── psi.py                  # Population Stability Index
│   │   ├── ks_test.py              # Kolmogorov-Smirnov
│   │   ├── js_divergence.py        # Jensen-Shannon
│   │   ├── wasserstein.py          # Wasserstein Distance
│   │   └── alert_system.py         # Sistema de alertas
│   │
│   ├── data/
│   │   ├── __init__.py
│   │   ├── loader.py               # Carregamento do PaySim
│   │   ├── preprocess.py           # Pré-processamento
│   │   ├── temporal_simulator.py   # Simulador temporal
│   │   └── drift_simulator.py      # Simulador de drift
│   │
│   ├── db/
│   │   ├── __init__.py
```

```
│   │   ├── database.py            # Conexão PostgreSQL
│   │   ├── models.py              # Modelos SQLAlchemy
│   │   └── migrations/            # Migrations Alembic
│   │
│   ├── experiments/
│   │   ├── e1_no_drift.py         # Experimento 1
│   │   ├── e2_drift_levels.py     # Experimento 2
│   │   ├── e3_detection_methods.py # Experimento 3
│   │   └── e4_retraining.py       # Experimento 4
│   │
│   ├── requirements.txt
│   └── Dockerfile
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── MetricsChart.tsx
│   │   │   ├── DriftChart.tsx
│   │   │   ├── AlertPanel.tsx
│   │   │   └── FeatureDrift.tsx
│   │   ├── pages/
│   │   │   ├── index.tsx
│   │   │   ├── experiments.tsx
│   │   │   └── settings.tsx
│   │   ├── services/
│   │   │   └── api.ts             # Chamadas à API
│   │   └── styles/
│   ├── package.json
│   └── Dockerfile
│
├── notebooks/
│   ├── 01_eda.ipynb               # Análise exploratória
│   ├── 02_baseline_model.ipynb    # Modelo baseline
│   ├── 03_drift_simulation.ipynb  # Simulação de drift
│   └── 04_results_analysis.ipynb  # Análise dos resultados
│
├── docker-compose.yml
├── README.md
└── .gitignore
```

🛠️ STACK TECNOLÓGICA COMPLETA

Camada	Tecnologia	Justificativa
Linguagem	Python 3.11+	Ecossistema de ML e estatística
ML	XGBoost, Scikit-learn	Robustez em dados tabulares
Estatística	SciPy, NumPy	Implementação de métricas
Drift	Evidently, implementações próprias	Flexibilidade e controle
API	FastAPI	Performance e documentação automática
Banco	PostgreSQL	Confiabilidade e suporte a JSON
ORM	SQLAlchemy	Integração com Python
Frontend	React/Next.js	Componentização e SSR
Gráficos	Recharts/Chart.js	Visualizações interativas
Container	Docker + Docker Compose	Reprodutibilidade
Versionamento	Git + GitHub	Colaboração
Documentação	Markdown + Swagger	Clareza

📊 ENTREGÁVEIS FINAIS

Entregável	Descrição	Responsável
D1	Dataset processado e simulado	Pessoa 1
D2	Modelo baseline treinado	Pessoa 1
D3	Módulos de detecção de drift	Pessoa 2
D4	API REST funcional	Pessoa 3
D5	Dashboard interativo	Pessoa 3

D6	Resultados dos 4 experimentos	Todas
D7	Artigo científico	Todas
D8	Documentação técnica	Pessoa 3
D9	Repositório GitHub	Todas

☑ PRÓXIMOS PASSOS IMEDIATOS

1. **Reunião de kickoff** para alinhar responsabilidades e prazos
2. **Configuração do repositório** GitHub com estrutura de diretórios
3. **Configuração do ambiente** Docker para desenvolvimento
4. **Download e análise** do dataset PaySim
5. **Implementação do modelo baseline** para validação inicial

Quer que eu detalhe alguma parte específica, como os **experimentos científicos**, a **implementação dos métodos de drift** ou o **dashboard**? Posso também criar os **diagramas de fluxo** de cada experimento. 🚀